

Assignment 3 — Bridge Pattern

Drawing Application — Option A

Course	ShP-2216 Software Design Patterns
Student	YELBAYEVA ZHULDYZ
Group	SE-2522
Repository / Commit	https://github.com/zhuldyzelbayeva-spec/SDP-3-assignmnet.git 5683204c06f5e99b51d7e192737486cb3f91c9a3

1. Design and Bridge Rationale

Application. The application demonstrates the Bridge Design Pattern for drawing shapes. The abstraction dimension is the shape hierarchy: Shape, Circle, and Square. The implementation dimension is the renderer hierarchy: Renderer, VectorRenderer, RasterRenderer, and AsciiRenderer.

Two independent dimensions. Shape classes contain domain data such as Circle.radius and Square.side, while renderer classes provide the low-level rendering behavior. Shape stores a reference to the Renderer interface and delegates rendering through that interface.

Why Bridge fits. The two dimensions can vary independently. A new renderer can be added without changing Circle or Square, and a new shape does not require renderer-specific subclasses. The same Circle object can also replace its renderer at runtime through setImplementation(...), which is demonstrated by T5.

2. UML Class Diagram

Figure 1. Final Bridge design with both hierarchies and the interface-typed bridge link.

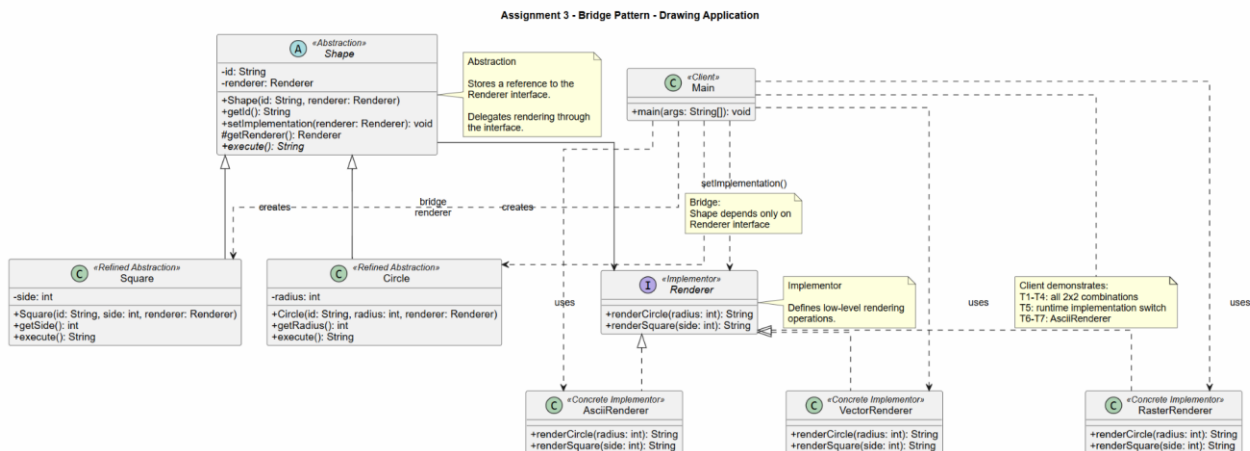


Diagram interpretation. Shape is the Abstraction; Circle and Square are Refined Abstractions. Renderer is the Implementor; VectorRenderer, RasterRenderer, and AsciiRenderer are Concrete Implementors. Main is the Client. The solid directed association from Shape to Renderer represents the bridge reference renderer: Renderer is an interface type, not a concrete implementation.

3. Clean Code Evidence — Five Annotated Excerpts

3.1 Separation of responsibilities — Shape

```
private final String id;
private Renderer renderer;

public void setImplementation(Renderer renderer) {
    if (renderer == null) {
        throw new IllegalArgumentException("renderer must not be null");
    }
    this.renderer = renderer;
}
```

Benefit: Shape owns abstraction-side identity and the implementation reference. Renderer classes are responsible only for rendering. This keeps the two Bridge dimensions separate.

3.2 Meaningful names — domain and pattern roles

```
private final int radius;      // Circle
private final int side;       // Square
private Renderer renderer;     // Shape

public void setImplementation(Renderer renderer) { ... }
```

Benefit: Names such as radius, side, renderer, execute, and setImplementation directly describe the data and Bridge roles, making the design easier to review and defend.

3.3 Small focused methods — Circle

```
@Override
public String execute() {
    return getRenderer().renderCircle(radius);
}
```

Benefit: Circle.execute() performs one focused task: it delegates the rendering operation to the current Renderer. The abstraction does not contain renderer-specific logic.

3.4 No duplicated workflow / renderer-specific behavior

```
@Override
public String renderCircle(int radius) {
    return "VECTOR circle radius=" + radius;
}
```

Benefit: Each renderer owns its own output format. The shared test workflow in Main is centralized in check(...), so the same PASS/FAIL logic is not duplicated across T1–T4, T6, and T7.

3.5 Encapsulated state and runtime switching — Main

```
Shape originalReference = circle;
String before = circle.execute();

circle.setImplementation(new RasterRenderer());

Shape afterReference = circle;
boolean sameObject = originalReference == afterReference;
```

Benefit: The object reference remains the same while the implementation reference changes. Circle's ID and radius are preserved, while execute() produces the result of the new renderer. This directly demonstrates T5.

4. I3 Extension and Design Trade-off

I3 extension. After the working I1/I2 two-by-two version, AsciiRenderer was added as the third Concrete Implementor. Main was extended only to demonstrate T6 and T7. The existing Shape, Circle, Square, Renderer, VectorRenderer, and RasterRenderer classes remain unchanged. The extension therefore adds a new implementation without modifying the existing abstraction hierarchy or the existing I1/I2 implementations.

Trade-off. Bridge increases structural complexity because it introduces two hierarchies and an implementor interface. For a very small application this means more classes and interfaces. The benefit is independent variation: new renderers can be introduced without creating renderer-specific shape subclasses.

5. Bridge versus Adapter

Bridge. Its intent is to separate an abstraction from its implementation so that both can vary independently. This is the relationship used here between Shape and Renderer.

Adapter. Its intent is to make an existing incompatible interface usable through an interface expected by a client, normally by translating one interface into another. This application does not translate an incompatible existing service; Renderer is the implementation contract designed for Shape. Therefore the design demonstrates Bridge.

6. Reproducibility and Evidence

Build and run commands: javac --release 17 -encoding UTF-8 -d out "@sources.txt" and java -cp out Main --demo

Required demonstration. T1–T4 demonstrate the four Circle/Square × Vector/Raster combinations. T5 demonstrates runtime switching on the same object. T6–T7 demonstrate the new AsciiRenderer implementation. The submitted demo output should record the actual results and the computed summary.

References

- [1] Astana IT University, School of Software Engineering. Assignment 3 | Bridge Pattern, ShP-2216 Software Design Patterns, 2026–2027.
- [2] E. Gamma, R. Helm, R. Johnson, J. Vlissides. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.

7. Actual Demo Evidence

The submitted demo-output.txt records the actual execution of all seven checks. The Java 17 build and demo were verified with the required commands. The final observed result was SUMMARY: 7/7 PASS.